OFFSIDE LABS

DAMM v2

Release 0.2.5

Smart Contract Security Assessment

October 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **5**

4 Key Findings and Recommendations **6**

4.1 Informational and Undetermined Issues 6

5 Disclaimer **8**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DAMM v2* smart contracts, starting on September 29th, 2026, and concluding on October 12th, 2026.

Project Overview

Meteora DAMM v2 Release 0.2.5 revises fee and reward accounting to prevent accumulator overflow from permanently blocking swaps, liquidity operations, and claims. Cumulative values now wrap, while pending position fees and rewards are capped at the supported token amount limit. This release also allows compounding pool configurations to specify a zero compounding allocation.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the *DAMM v2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Release 0.2.5: `PR-229`
  - Commit Hash: 8c89b9e5407517ae5ee69103a0b38b8a2c438cc6

The issues described in this report have been resolved by the following commit:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Commit Hash: 8c89b9e5407517ae5ee69103a0b38b8a2c438cc6

We listed the files we have audited below:

- DAMM v2 Release 0.2.5
  - programs/cp-amm/src/instructions/ix_claim_position_fee.rs
  - programs/cp-amm/src/instructions/ix_withdraw_dead_liquidity_reward.rs
  - programs/cp-amm/src/math/safe_math.rs
  - programs/cp-amm/src/math/u128x128_math.rs
  - programs/cp-amm/src/math/utils_math.rs
  - programs/cp-amm/src/params/fee_parameters.rs
  - programs/cp-amm/src/state/pool.rs
  - programs/cp-amm/src/state/position.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 0 low issue
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Restricted Shift Counts in Arithmetic Helpers	Informational	Acknowledged
02	Inconsistent Error Handling Between Rounding Modes	Informational	Acknowledged

4 Key Findings and Recommendations

4.1 Informational and Undetermined Issues

Restricted Shift Counts in Arithmetic Helpers

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Code QA

`calculate_position_fee_or_reward` uses a U384 intermediate, while `mul_shr_256` and `mul_shr_256_wrapping_u64` use U512 intermediates.

However, their `offset` parameters are `u8`, restricting shifts to 0–255. These interfaces therefore cannot express otherwise valid right shifts of 256–383 or 256–511, respectively. Current program callers use fixed offsets of 128 or 192, so existing calculations are unaffected.

It's recommended to use `u16` or `usize` for these offset parameters and explicitly validate that each shift is smaller than the intermediate type's bit width.

Inconsistent Error Handling Between Rounding Modes

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Code QA

In `safe_mul_div_cast_u128`, the two rounding modes handle a zero denominator differently.

```
45      let prod = U256::from(x).safe_mul(U256::from(y))?;
46      let denominator = U256::from(denominator);
47
48      let result = match rounding {
49          Rounding::Up => prod.div_ceil(denominator),
50          Rounding::Down => prod.safe_div(denominator)?,
51      };
```

programs/cp-amm/src/math/utils_math.rs#L45-L51

`Rounding::Down` calls `safe_div`, which uses `checked_div` and converts its `None` result into `PoolError::MathOverflow`.

```
61              fn safe_div(self, v: $t) -> Result<$t, PoolError> {
62                  match self.checked_div(v) {
63                      Some(result) => Ok(result),
64                      None => {
65                          let caller = Location::caller();
66                          msg!("Math error thrown at {}:{}", caller.file(),
                                ↪ caller.line());
67                          Err(PoolError::MathOverflow)
68                      }
```

programs/cp-amm/src/math/safe_math.rs#L61-L68

However, `Rounding::Up` directly calls `U256::div_ceil`, which panics on division by zero. The function's Result return type does not convert this panic into an error.

```
35      pub fn div_ceil(self, rhs: Self) -> Self {
36          let (q, r) = self.div_rem(rhs);
37          if r.is_zero() {
38              q
39          } else {
40              q + Self::ONE
41          }
42      }
```

src/div.rs#L35-L42

```
52      pub fn div_rem(mut self, mut rhs: Self) -> (Self, Self) {
53          algorithms::div(&mut self.limbs, &mut rhs.limbs);
54          (self, rhs)
55      }
```

src/div.rs#L52-L55

```
48  pub fn div(numerator: &mut [u64], divisor: &mut [u64]) {
49      // Trim most significant zeros from divisor.
50      let i = divisor
51          .iter()
52          .rposition(|&x| x != 0)
53          .expect("Divisor is zero");
```

src/algorithms/div/mod.rs#L48-L53

Consequently, the same invalid denominator produces either a returned error or a panic depending on the rounding mode.

It's recommended to explicitly reject a zero denominator with `PoolError` before selecting the rounding branch, ensuring consistent error handling in both modes.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

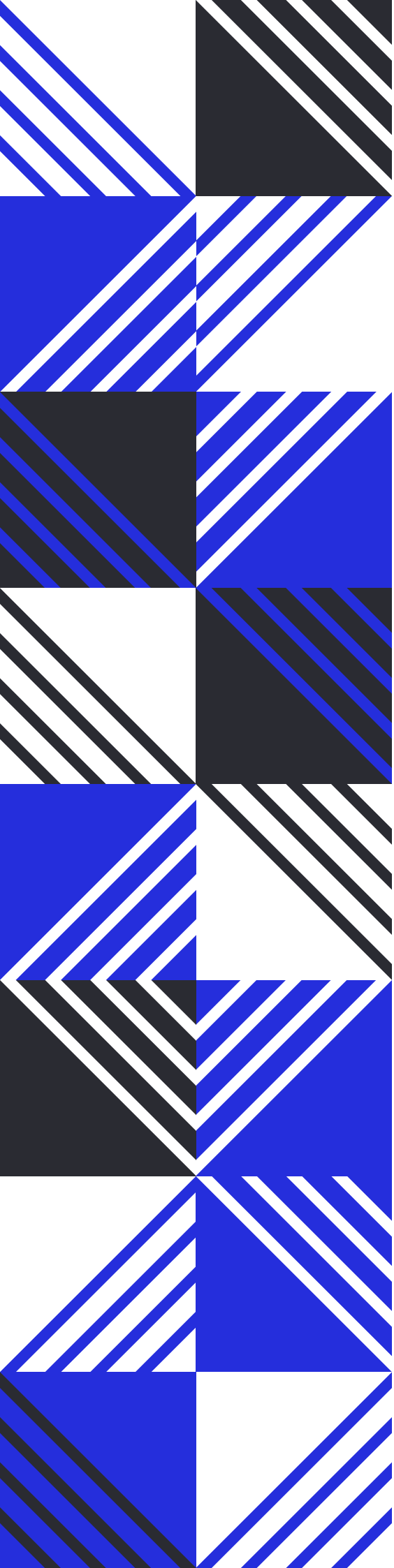
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs